

SC4023 AI for Software Engineering
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumsclaw/ntu-cs-textbooks

Contents

- 1 Introduction: Where AI Meets Software Engineering 1**
 - 1.1 What Is Software Engineering? 1
 - 1.2 Why Software Engineering Needs AI 2
 - 1.3 A Taxonomy of AI for SE 3
 - 1.4 Two Directions: AI for SE and SE for AI 4
 - 1.5 Challenges, Limits, and How to Read This Book 5
 - 1.6 Chapter Notes 5
 - 1.7 Exercises 6
- 2 Code Generation with Large Language Models 7**
 - 2.1 How a Model Learns to Write Code 7
 - 2.2 Prompting: Talking to the Model 8
 - 2.3 GitHub Copilot in Practice 9
 - 2.4 Evaluation: HumanEval and What It Misses 10
 - 2.5 Chapter Notes 11
 - 2.6 Exercises 11
- 3 AI for Testing: Generation, Fuzzing, and Oracles 12**
 - 3.1 Why Testing Is Hard: Coverage and Oracles 12
 - 3.2 AI-Based Test Generation 13
 - 3.3 Fuzzing with Intelligence 14
 - 3.4 The Oracle Problem and How to Sidestep It 15
 - 3.5 Chapter Notes 15
 - 3.6 Exercises 16

4	Automated Program Repair	17
4.1	What Is Automated Program Repair?	17
4.2	GenProg and Search-Based Repair	18
4.3	Learning Fix Patterns: From TBar to SequenceR	19
4.4	Evaluation, Overfitting, and Limits	20
4.5	Chapter Notes	21
4.6	Exercises	21
5	Code Quality and Intelligent Review	23
5.1	Intuition: What Is “Good” Code?	23
5.2	Measuring Quality: Metrics that Matter	23
5.3	Static Analysis: Traditional to Learning-Augmented	24
5.4	AI-Assisted Code Review	25
5.5	Quality Gates and Developer Experience	26
5.6	Worked Example: From Metric to Action	26
5.7	Common Pitfalls	27
5.8	Chapter Summary	27
5.9	Exercises	27
6	Natural Language for Requirements	29
6.1	Intuition: Why Requirements Are Hard	29
6.2	Ambiguity: A Taxonomy You Can Act On	30
6.3	NLP Pipeline for Requirements	30
6.4	Traceability: Linking Wishes to Code and Tests	31
6.5	Putting It Together: An RE Assistant	32
6.6	Worked Example: End-to-End on a Sprint	33
6.7	Common Pitfalls	33
6.8	Chapter Summary	34
6.9	Exercises	34
7	AI in DevOps: CI/CD, Defects, and Estimation	35

7.1	Intuition: The DevOps Loop	35
7.2	AI Across the Pipeline	36
7.3	Defect Prediction: Will This Change Break?	36
7.4	Effort Estimation: How Long Will It Take?	38
7.5	Worked Example: Prioritising Tests with History	38
7.6	Common Pitfalls	39
7.7	Closing the Loop: Measurement that Matters	39
7.8	Chapter Summary	39
7.9	Exercises	40
8	The Future: Ethics, Collaboration, and Frontiers	41
8.1	Intuition: Why “Just Ship It” Is Not Enough	41
8.2	Ethics in AI-Generated Code	41
8.3	Intellectual Property and Licensing	42
8.4	Human–AI Collaboration: Augmentation, Not Replacement	43
8.5	Research Frontiers	43
8.6	Putting Ethics into Practice: A Checklist	44
8.7	Chapter Summary	45
8.8	Exercises	45

Chapter 1

Introduction: Where AI Meets Software Engineering

“Programs must be written for people to read, and only incidentally for machines to execute.” — Harold Abelson. When programs grew from hundreds to millions of lines, reading, writing, and fixing them became a human bottleneck. AI now offers to read, write, and fix alongside us.

From zero: no AI or SE background assumed. We build from everyday intuition—what it means to build reliable software, why it is hard, and where a learning machine can help—before any model or metric. Every term is defined on first use, picture first, then formalism. This chapter is the map for the whole book.

Learning Outcomes

After this chapter you will be able to:

- (L1) describe the software engineering lifecycle and name its core phases and artefacts;
- (L2) explain why scale, complexity, and human factors make SE hard and costly;
- (L3) classify AI-for-SE tasks (generation, testing, repair, requirements, management);
- (L4) distinguish the direction AI→SE from SE→AI and give examples of each;
- (L5) state the key challenges—data quality, evaluation, and trust—that bound what AI can do.

1.1 What Is Software Engineering?

Intuition. Imagine building a bridge. You do not start by welding steel; you survey the river, sketch designs, estimate load, pour foundations, test every joint, and schedule inspections for decades. Software is similar, but the bridge is invisible, changes weekly, and must carry millions of users at once. *Software engineering* (SE) is the disciplined process for building that invisible bridge so it does not collapse under its own weight.

Formal view. The *software development lifecycle* (SDLC) organises work into phases: *requirements* (what to build), *design* (how to structure it), *implementation* (writing code), *verification* (testing and review), and *maintenance* (fixing and evolving). Each phase produces *artefacts*: specifications, architecture diagrams, source code, test suites, and issue reports. A *process model* (waterfall, agile, DevOps) orders and iterates these phases.

Definition 1.1 (Software engineering). *Software engineering* is the systematic application of engineering principles—abstraction, decomposition, measurement, and process—to the design, construction, verification, and evolution of software systems under constraints of time, cost, and quality.

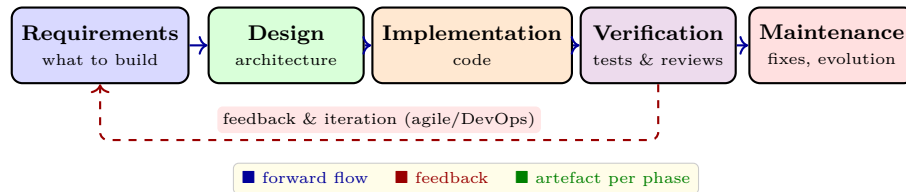


Figure 1.1: The SE lifecycle as a pipeline with feedback. Each box produces an artefact; arrows are hand-offs. Modern processes iterate rapidly around this loop rather than traversing it once.

Each hand-off is a translation: natural-language needs become specifications, specifications become code, code becomes behaviour checked by tests. Every translation can introduce defects, and defects become exponentially costlier when discovered late. That cost curve is the economic reason SE needs help.

An old debate contrasts *plan-driven* (waterfall: freeze requirements, then design and build) with *iterative* (agile/DevOps: short sprints, continuous feedback). In practice, both produce the same artefacts; they differ in batch size and how quickly feedback loops close. AI assistance is valuable in either model because it shortens each translation and tightens each loop.

1.2 Why Software Engineering Needs AI

Intuition. A single developer can hold a few thousand lines in mind. A modern product holds tens of millions, changes hundreds of times per day, and is touched by hundreds of contributors across time zones. No human can see the whole system at once, yet every change can break it. AI helps because much of SE work is *patterned*: similar bugs recur, similar tests are needed, similar code is written again and again.

Three forces create the need. First, *scale*: codebases, logs, and issue trackers generate far more data than any team can read. Second, *complexity*: interactions between components are combinatorial and non-local; a one-line change can fail a distant test. Third, *human limits*: fatigue, context switching, and incomplete knowledge cause mistakes. AI, trained on millions of prior examples, can surface relevant patterns fast, draft candidates, and flag risks—not replacing judgement but amplifying it.

Example 1.1 (The cost-of-delay curve). Boehm’s classic data shows a defect found in requirements costs 1× to fix, in design 5×, in coding 10×, in testing 50×, and in production 100–1000×. Any tool that moves detection one phase earlier pays for itself, even if it is imperfect. AI code review that catches a null dereference at write-time saves the production incident that would have followed.

A complementary view is *cognitive load*: developers spend roughly 60% of time comprehending code, not writing it. Tools that summarise, navigate, or explain code reduce that load directly, even when they do not generate a single line.

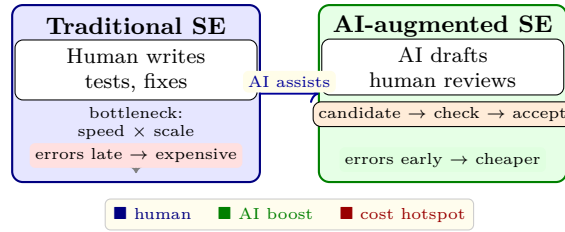


Figure 1.2: Why AI helps: the traditional bottleneck shifts when AI drafts and humans review, catching defects earlier where fixes are cheapest.

A concrete slice: from issue to patch. Consider a real loop: a user files issue “app crashes when name is empty,” a developer reproduces it, localises the null dereference, writes a guard, adds a test, and opens a pull request. Each step involves reading artefacts (issue text, stack trace, surrounding code), deciding, and writing. An AI assistant can help at every step: summarise the issue, suggest the suspicious lines, draft the guard and the test, and explain the change in the PR description. The human still decides, but the loop is shorter.

Definition 1.2 (SE artefact). An *artefact* is any persistent product of the lifecycle: a requirement sentence, a design diagram, a source file, a test case, a bug report, or a log trace. AI-for-SE treats artefacts as data: it learns joint distributions over them and predicts missing or next artefacts given observed ones.

1.3 A Taxonomy of AI for SE

Intuition. Not all AI help looks the same. Sometimes AI *writes* (code, tests, docs); sometimes it *checks* (finds bugs, predicts risk); sometimes it *decides* (prioritises issues, assigns reviewers). A simple map prevents confusion about what is being automated and how to evaluate it.

We organise AI-for-SE along two axes: *artefact* (requirements, code, tests, defects, process) and *role* (generate, analyse, predict, assist). This yields five active fronts: *code generation* (Ch. 2), *test generation* (Ch. 3), *program repair* (Ch. 4), *requirements and comprehension*, and *management and prediction* (effort, review, triage). A sixth, *SE for AI*, flips the direction—how SE practices make AI systems reliable, tested, and maintainable.

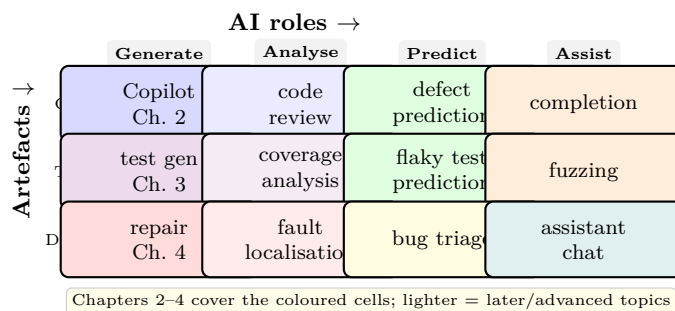


Figure 1.3: Taxonomy of AI-for-SE. Rows are artefacts, columns are roles. Colour groups related tasks; this book focuses on the three most mature code-adjacent cells.

Listing 1.1: Mining a tiny SE signal: does churn predict defects? A toy illustration of the data AI learns from.

```

1 # Each commit: lines changed (churn) and whether it introduced a bug (0/1)
2 commits = [(120,0),(800,1),(45,0),(600,1),(300,0),(950,1),(20,0)]
3 threshold = 500 # simple rule: high churn -> flag for review
4 for churn, bug in commits:
5     flag = "REVIEW" if churn > threshold else "ok"

```

```

6     print(f"churn={churn:3d}  bug={bug}  -> {flag}")
7 # Real models use dozens of features (author, files, time, history).
8 # The point: SE artefacts are data; patterns in them are learnable.

```

The same data view applies everywhere: requirements need ambiguity detection, code needs completion, tests need generation, and issues need triage. Different models (LLMs, classifiers, search) fit different cells, but the workflow is shared—learn from history, propose, let humans decide.

Listing 1.2: From pattern to assistant: the AI-for-SE loop in miniature.

```

1 def ai_for_se_loop(history, new_task):
2     """History = list of (artefact, outcome). New task = current artefact."""
3     patterns = learn_patterns(history)      # e.g., frequent bug patterns
4     candidates = generate(patterns, new_task) # draft code/tests/fixes
5     ranked = rank_by_confidence(candidates) # model scores
6     # Human in the loop: review, test, accept or reject
7     return [c for c in ranked if human_review(c)]
8
9 def learn_patterns(history): return {"null-check": 0.9, "off-by-one": 0.7}
10 def generate(patterns, task): return [f"candidate for {task} via {p}" for p in patterns]
11 def rank_by_confidence(cs): return sorted(cs, reverse=True)
12 def human_review(c): return True # stub: real review runs tests + inspection

```

1.4 Two Directions: AI for SE and SE for AI

Intuition. There are two streets between AI and SE, and they carry traffic in opposite directions. *AI for SE* uses learning to help build software (the focus of this book). *SE for AI* uses engineering discipline to make AI systems themselves reliable. Confusing the two leads to mismatched expectations—a great code generator still needs versioning, testing, and monitoring like any other component.

Examples clarify. AI-for-SE: Copilot completes a function, a classifier predicts which commit will be buggy, a repair bot drafts a patch. SE-for-AI: data versioning for training sets, model regression tests that fail when accuracy drops, canary deployments for an LLM service, and experiment tracking that reproduces a model from six months ago. The latter is not optional: without it, “it worked on my notebook” becomes the new “it worked on my machine.”

Definition 1.3 (AI for SE vs. SE for AI). *AI for SE* applies AI techniques to SE tasks (code, test, repair, review). *SE for AI* applies SE techniques to AI systems (data pipelines, model testing, deployment, monitoring, and governance). Modern products need both, but this textbook emphasises the first while noting where the second is required.

A simple check: if the artefact being produced is traditional software (a feature, a test, a fix), you are doing AI-for-SE; if the artefact is a model, dataset, or inference service that must itself be engineered, you are doing SE-for-AI. Teams that master both ship faster and break less.

1.5 Challenges, Limits, and How to Read This Book

What AI cannot do matters as much as what it can. Three limits recur. *Data quality*: training on public code inherits bugs, secrets, and biases; garbage in, garbage out. Duplicated or leaked test data inflates benchmark scores while hiding real weaknesses. *Evaluation*: passing a benchmark (e.g., HumanEval) does not mean the tool helps on your codebase, with your tests, under your deadlines. Lab metrics and field utility diverge. *Trust and accountability*: generated code must be reviewed, tested, and owned; an accepted suggestion is still the developer’s responsibility. Ethical and legal questions—licensing of training data, leakage of credentials, over-reliance, and deskilling—sit on top of technical ones.

A practical mental model is *assistant, not autopilot*: AI proposes, humans dispose. Every chapter therefore pairs a generation capability with its checking counterpart—how to test generated code (Ch. 3), how to validate a repair (Ch. 4), and how to measure whether the help actually helped (productivity, quality, and risk metrics introduced next). The field moves fast, but the checking principles—specifications, tests, and careful measurement—are durable.

How to use this book. Each chapter follows intuition → formal definition → worked example → proof or analysis → exercises. Chapters 2–4 are the technical core (generation, testing, repair) and can be read independently after this introduction, though they share vocabulary introduced here. Exercises include both conceptual checks and small hands-on prompts you can run with any public LLM or IDE assistant. When you try a prompt, record what you gave, what you got, and how you checked it—that habit is the core skill the book teaches.

1.6 Chapter Notes

Further reading: Pressman and Maxim’s *Software Engineering: A Practitioner’s Approach* for the lifecycle; Boehm (1981) for the cost-of-change curve; Harman et al. (2015) for the vision of AI-for-SE; recent surveys by Watson et al. (2022) and Fan et al. (2023) for the generation/testing/repair landscape. Historical note: the term “software engineering” was coined at the 1968 NATO conference to argue that building software needed the discipline of other engineering fields—the same argument now motivates AI assistance.

Remark 1.1 (Reading tip). When you encounter an AI-SE claim (e.g., “tool X improves productivity by 55%”), ask three questions: What was the task? What was the baseline? What was the measure? The same tool can look brilliant on completion benchmarks and modest on bug-fix rates. Chapters 2–4 train you to ask those questions for generation, testing, and repair respectively.

A note on terminology: we use *AI*, *ML*, and *LLM* loosely in this introduction. Later chapters are precise: Ch. 2 distinguishes language models from code models and prompting from fine-tuning; Ch. 3 distinguishes search-based from learning-based test generation; Ch. 4 distinguishes heuristic repair from learned repair.

1.7 Exercises

- E1.1 **Lifecycle mapping.** Pick a familiar app (e.g., a campus food-ordering app). List one concrete artefact for each SDLC phase and one feedback arrow from Fig. 1.1 that would fire when a late requirement changes. Explain why the feedback costs more than catching it early.
- E1.2 **Cost curve.** A defect costs \$100 if found in requirements, \$500 in design, \$1,000 in coding, \$5,000 in testing, and \$50,000 in production. A team finds 20 defects, evenly spread across phases. Compute total cost. If an AI review tool moves half the production defects to coding, what is the saving?
- E1.3 **Taxonomy placement.** For each task, place it in Fig. 1.3: (a) summarising a pull request, (b) predicting which files are most likely to contain bugs next release, (c) generating JUnit tests from a method signature, (d) auto-fixing a null-pointer crash. State the row and column and justify in one sentence.
- E1.4 **AI→SE vs. SE→AI.** Give one example of AI helping SE and one of SE helping AI. For the second, explain which SE practice (testing, versioning, or monitoring) matters most for a deployed LLM service and why retraining alone is insufficient.
- E1.5 **Trust and evaluation.** A code-generation model scores 75% on HumanEval but your team observes only 30% of its suggestions are accepted. Propose two distinct hypotheses for the gap and design a one-week A/B experiment on your repository that would distinguish them.

Chapter 2

Code Generation with Large Language Models

“The best way to predict the future is to invent it.” — Alan Kay. Code-generating models try to do both at once: they predict the next token and thereby invent the program that follows your intent.

From zero: no LLM background assumed. We start from an empty file and a cursor: what does it mean to predict code, how does a model learn that skill from millions of repositories, and how do you talk to it so it helps rather than hallucinates? Every term—token, prompt, `pass@k`—is built picture-first, then defined.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain how LLMs are trained on code (tokens, next-token loss, context window);
- (L2) craft prompts using zero-shot, few-shot, and chain-of-thought patterns;
- (L3) describe how GitHub Copilot turns cursor context into suggestions;
- (L4) compute `pass@k` on HumanEval and interpret what it measures;
- (L5) state failure modes (hallucination, leakage, insecurity) and mitigations.

2.1 How a Model Learns to Write Code

Intuition. A junior developer learns by reading thousands of files: they see that after `for i in range(` the next characters are usually `len(arr))`, and after `def test_` the next lines often assert something. An LLM learns the same regularities, but at scale—over billions of tokens—and stores them as weights that predict what comes next.

Formally, source code is split into *tokens* (roughly words or subwords). Given a prefix $x_{1:t-1}$, the model learns

a distribution $P_\theta(x_t | x_{1:t-1})$ and is trained to minimise *cross-entropy* next-token loss on a large code corpus:

$$\mathcal{L}(\theta) = - \sum_t \log P_\theta(x_t | x_{1:t-1}).$$

A *Transformer* implements P_θ with self-attention: each position attends to all previous positions, so the prediction for the next token sees the full prefix (prompt + file context + cursor). Pretraining on raw code gives a *base model*; fine-tuning on prompt–response pairs (instruction tuning) and on accepted edits (RLHF or direct preference) steers it toward helpful completions.

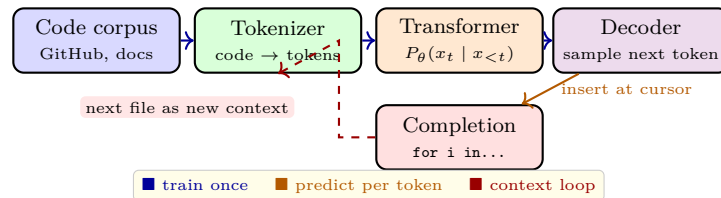


Figure 2.1: LLM for code as next-token prediction. Training minimises $-\log P_\theta$ over a tokenised corpus; at inference the model samples tokens one by one conditioned on the prompt and surrounding file context.

Two limits matter immediately. The *context window* (e.g., 8k–128k tokens) caps how much surrounding code the model can see; beyond it, information is truncated. *Exposure bias* means training sees ground-truth prefixes but generation feeds its own past samples, so errors can compound.

Example 2.1 (Next-token intuition). Prefix: `def fib(n):\n if n <= 1:\n`. The model assigns high probability to `return n` and to `return 1`, lower to `print(`. Sampling picks one; temperature controls how adventurous the pick is.

2.2 Prompting: Talking to the Model

Intuition. A prompt is not a magic spell; it is a specification written in natural language plus examples. The same model can produce insecure, correct, or over-engineered code depending on how you frame the request. Prompt engineering is specification engineering with a probabilistic interpreter.

Three patterns cover most uses. *Zero-shot*: describe the task outright (“Write a Python function that returns the median of a list”). *Few-shot*: give 2–3 input–output examples before the real request, anchoring style and edge cases. *Chain-of-thought (CoT)*: ask the model to reason step by step before coding (“First list edge cases, then write the function”); this often reduces off-by-one and null-handling errors.

Listing 2.1: Three prompting styles for the same task. The code is the same; the prompt changes the result.

```

1 # Zero-shot: bare instruction
2 # Prompt: "Write a Python function median(xs) that handles empty and even-length lists."
3 def median_zero_shot(xs):
4     if not xs: return None
5     s = sorted(xs); n = len(s)
6     return s[n//2] if n % 2 == 1 else (s[n//2-1]+s[n//2])/2
7
8 # Few-shot: show examples, then the task (model imitates pattern)
9 # Example 1: median([3]) -> 3    Example 2: median([1,2,3,4]) -> 2.5
10 # Now complete: median([5,1,9]) -> ?

```

```

11
12 # Chain-of-thought prompt (in comment) -> model writes reasoning first
13 # "Steps: 1) handle empty, 2) sort, 3) if odd return middle else average two middles."
14 def median_cot(xs):
15     # step 1: empty guard
16     if not xs: return None
17     # step 2: sort
18     s = sorted(xs); n = len(s); mid = n//2
19     # step 3: branch on parity
20     return float(s[mid]) if n % 2 == 1 else (s[mid-1]+s[mid])/2.0

```

Practical tips: be explicit about language, I/O format, and edge cases; include a tiny example when the output shape matters; forbid unwanted behaviour (“do not use recursion”); and ask for tests alongside code. The model is sensitive to all of these.

Prompt sensitivity. Small rewordings can swing correctness: adding “handle empty input” may fix a null failure; swapping the order of examples may change which edge case the model copies. This brittleness is not a bug to patch but a property of probabilistic models: the prompt is the specification, and vague specifications yield variable implementations. Good prompts are precise, minimal, and testable.

Definition 2.1 (Prompt as specification). A *prompt* is a (possibly incomplete) specification of the desired program in natural language plus optional examples and constraints. Better prompts reduce the set of programs consistent with the specification, but they never reduce it to one—the model still samples.

2.3 GitHub Copilot in Practice

In the IDE, Copilot sees far more than your prompt: the open file, neighbouring files, cursor position, and recent edits. It ranks candidate completions by model score and by heuristics (how well they parse, whether they reuse identifiers in scope). The developer’s job shifts from writing every character to *reviewing* candidates—accept, edit, or dismiss.

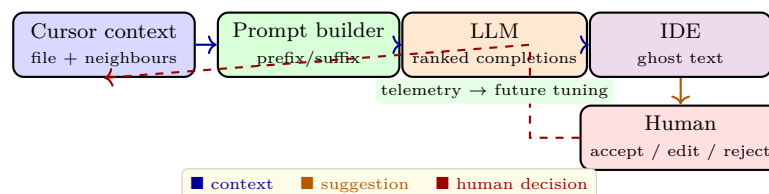


Figure 2.2: Copilot’s inline loop. Context is turned into a prompt, the model proposes ranked completions rendered as ghost text, and the developer decides. Telemetry (accept/reject/edit) later tunes ranking.

Empirical studies report 30–55% acceptance rates for shown completions and measurable speed-ups on boilerplate and test scaffolding, with smaller gains on novel algorithmic code. The risk is not just wrong code but *plausible* wrong code—it type-checks, looks idiomatic, and fails on an edge case the prompt omitted.

2.4 Evaluation: HumanEval and What It Misses

Intuition. To know whether a model is useful, you need a test where the answer is unambiguous. HumanEval provides that: 164 hand-written Python problems, each with a function signature, docstring, and hidden unit tests. The model generates code; we run the tests; it either passes or not.

The metric is $\text{pass}@k$: generate k samples per problem, count the problem as solved if *any* sample passes all tests. For n samples with c correct, the unbiased estimator used in practice is

$$\text{pass}@k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}, \quad \text{and } \text{pass}@1 \text{ is just accuracy when } k = 1.$$

Reporting $\text{pass}@1$, $\text{pass}@10$, and $\text{pass}@100$ shows the trade-off between single-try reliability and the value of re-sampling.

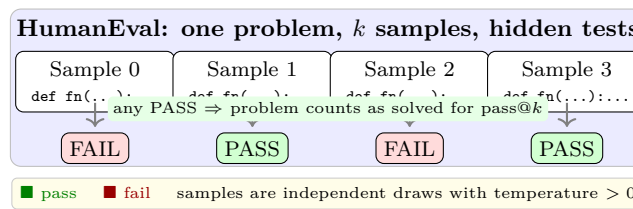


Figure 2.3: Computing $\text{pass}@k$. Generate k candidates per problem; the problem is solved if at least one candidate passes all hidden tests. Higher k rewards diversity, not just single-shot precision.

Listing 2.2: A HumanEval-style problem: the model must complete the function to pass hidden tests.

```

1 def has_close_elements(numbers, threshold):
2     """Check if any two numbers are closer than threshold."""
3     # Model must generate the body below to pass hidden tests
4     for i in range(len(numbers)):
5         for j in range(i+1, len(numbers)):
6             if abs(numbers[i] - numbers[j]) < threshold:
7                 return True
8     return False
9
10 # Hidden tests (not shown to model):
11 assert has_close_elements([1.0, 2.0, 3.0], 0.5) == False
12 assert has_close_elements([1.0, 2.8, 3.0], 0.3) == True
13 # pass@k: does any of k samples pass all asserts?

```

HumanEval is clean but narrow: single-function Python, no project context, no dependency on other files. Real utility needs broader suites—MBPP, DS-1000, SWE-Bench (full repository issues), and project-specific acceptance tests. Leakage (training data containing the benchmark) and prompt sensitivity further qualify any headline number.

Failure modes to watch. Three recur in practice. *Hallucination*: the model invents an API that does not exist (e.g., `df.to_csv_with_header()`) with confident syntax. *Insecurity*: it reproduces an unsafe pattern seen in training, such as string-interpolated SQL. *Non-determinism*: the same prompt yields different outputs across runs (controlled by temperature); a completion that passed yesterday may fail today. Mitigations are engineering, not hopes: pin dependencies, run tests on every suggestion, and treat generated code as untrusted

input.

Definition 2.2 (Temperature). *Temperature* $T > 0$ scales logits before softmax: lower T sharpens the distribution (more deterministic, more typical), higher T flattens it (more diverse, more surprising). For code, $T \approx 0.2$ favours pass@1; $T \approx 0.8$ favours pass@100 by trading precision for coverage.

Remark 2.1 (From benchmark to IDE). A 10-point gain in pass@1 does not guarantee a 10% productivity gain. Measure what you ship: acceptance rate, time to task, defect rate, and review comments. The benchmark tells you the model can code; your repository tells you whether it helps you code.

2.5 Chapter Notes

Chen et al. (2021, Codex/HumanEval) introduced pass@ k and the 164-problem HumanEval set; Austin et al. (MBPP) and Lai et al. (DS-1000) broaden it. GitHub’s Copilot reports and Ziegler et al. (2022) study acceptance and productivity. For prompting, see Wei et al. on chain-of-thought and recent code-specific prompt surveys. Security caveat: Pearce et al. (2022) show language models can reproduce vulnerable patterns; always test and scan generated code.

2.6 Exercises

- E2.1 **Tokens and context.** Tokenise `for i in range(len(xs)):` by hand into plausible tokens. If the model’s context window is 8 tokens and your file prefix is 20 tokens long, which tokens are dropped and why might that cause a wrong completion for a variable defined at the top of the file?
- E2.2 **Prompt design.** Write zero-shot, few-shot (2 examples), and chain-of-thought prompts for: “Return the k most frequent words in a text, case-insensitive, tie-broken alphabetically.” Run all three on the same model (any public LLM) and compare outputs on an empty string and on ties.
- E2.3 **pass@ k calculation.** For one HumanEval problem you draw $n = 10$ samples and $c = 3$ pass. Compute pass@1, pass@2, and pass@5 using the formula in §2.4. Explain why pass@10 would be 1.0 in this setting and what that reveals about the metric.
- E2.4 **Copilot loop.** Sketch the data that Copilot sends as context when your cursor is inside a function that calls an internal API `db.query(sql)`. What could go wrong if a secret appears in a neighbouring file? Propose one mitigation at the prompt-building stage and one at the review stage.
- E2.5 **Beyond HumanEval.** A model scores 72% pass@1 on HumanEval but only 18% on SWE-Bench (repository-level issues). Give three concrete reasons for the gap that are not “the model got worse.” Propose one evaluation you would run on your own codebase before adopting the model.

Chapter 3

AI for Testing: Generation, Fuzzing, and Oracles

“Testing shows the presence, not the absence of bugs.” — Edsger Dijkstra. AI does not change that truth, but it can search for presence far faster and in corners humans rarely look.

From zero: no testing background assumed. We start from a single function and the question “did it do what I meant?”—what a test is, what an oracle is, and why writing both by hand does not scale. Every idea appears picture-first, then as a definition you can apply.

Learning Outcomes

After this chapter you will be able to:

- (L1) define test cases, oracles, and coverage (line, branch, mutation);
- (L2) explain why the oracle problem limits automated testing;
- (L3) describe search-based and LLM-based test-generation techniques;
- (L4) sketch an AI-guided fuzzing loop and its feedback signals;
- (L5) apply metamorphic and differential oracles to sidestep missing specifications.

3.1 Why Testing Is Hard: Coverage and Oracles

Intuition. To test a function you need two things: an *input* to run it on and a way to judge the *output*. The first is easy to invent; the second is where meaning lives. “Does `sort([3,1,2])` return `[1,2,3]`?” is a useful oracle. “Does this web handler behave correctly on all edge cases?” is a question humans struggle to answer completely.

Formally, a *test case* is a triple (x, y, check) where x is input, y is expected output (or property), and **check**

decides pass/fail. The *oracle problem* is that y is often unspecified, expensive to compute, or itself buggy. Without an oracle, execution is just a run—not a test.

Coverage measures how much of the program a suite exercises. *Line coverage* is the fraction of executable lines run; *branch coverage* is the fraction of `if/switch` edges taken. Higher coverage does not imply correctness, but low coverage guarantees untested behaviour.

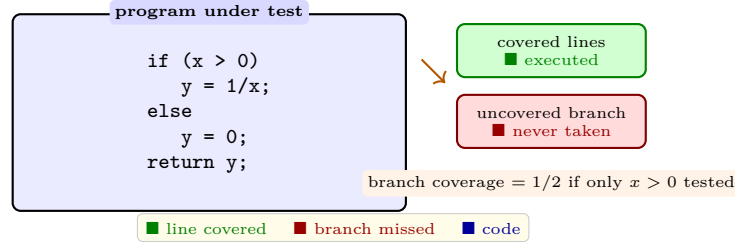


Figure 3.1: Coverage intuition: running tests lights up lines and branches. Line coverage can be 100% while branch coverage is 50%, leaving the `else` path untested and its bug hidden.

Two other useful measures: *mutation score* (fraction of seeded artificial faults a suite catches) and *path coverage* (fraction of distinct control-flow paths). Both are stronger but far more expensive.

Definition 3.1 (Oracle). An *oracle* is a procedure $O(x, P(x))$ that returns pass/fail for input x and program output $P(x)$. Examples: a golden output y^* (exact oracle), an assertion $x.length \geq 0$ (partial oracle), or a relation between two outputs (metamorphic oracle, §3.4).

3.2 AI-Based Test Generation

Intuition. Humans write tests one by one. Machines can propose hundreds, keep those that increase coverage or kill mutants, and discard the rest. The pattern is *generate–execute–filter*: draft candidates, run them, keep what helps.

Two families dominate. *Search-based* tools (e.g., EvoSuite for Java) evolve test suites with a genetic algorithm: fitness is coverage plus mutation kill, mutation adds branches, crossover mixes tests. *LLM-based* tools prompt a code model with the method signature, docstring, and surrounding code and ask it to “write JUnit tests for `median`.” The LLM brings natural-language understanding; search brings systematic coverage.

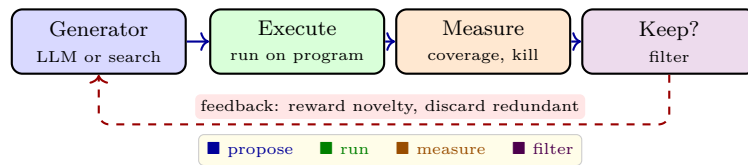


Figure 3.2: Generate–execute–filter loop. Both EvoSuite and LLM-based generators iterate it; the difference is how candidates are proposed (evolution vs. sampling).

Listing 3.1: Prompting an LLM to generate tests: give it the function and ask for edge cases.

```

1 # Function under test
2 def median(xs):
3     if not xs: return None
4     s = sorted(xs); n = len(s)
5     return s[n//2] if n % 2 == 1 else (s[n//2-1]+s[n//2])/2

```

```

6
7 # Prompt to LLM (in practice, include file context + docstring):
8 # "Write pytest tests for median covering empty, odd, even, duplicates, negatives."
9 def test_median():
10     assert median([]) is None          # empty
11     assert median([3]) == 3            # single
12     assert median([1,2,3]) == 2        # odd
13     assert median([1,2,3,4]) == 2.5    # even
14     assert median([2,2,2]) == 2        # duplicates
15     assert median([-5,-1,0]) == -1     # negatives
16 # LLM drafts; human (or runner) filters: do they compile? do they increase coverage?

```

LLM tests are often readable and cover natural edge cases humans forget (empty, null, duplicate), but they can also assert wrong oracles—the model may “learn” the bug as intended behaviour. Always run generated tests against a known-good version when available.

Listing 3.2: LLM-generated JUnit test (Java) for the same median logic. The oracle is explicit.

```

1 import static org.junit.Assert.*;
2 import org.junit.Test;
3 public class MedianTest {
4     @Test public void testEven() {
5         assertEquals(2.5, Median.median(new int[]{1,2,3,4}), 1e-9);
6     }
7     @Test public void testEmpty() {
8         assertNull(Median.median(new int[]{}));
9     }
10    @Test public void testDuplicates() {
11        assertEquals(2.0, Median.median(new int[]{2,2,2}), 1e-9);
12    }
13 }

```

3.3 Fuzzing with Intelligence

Intuition. If you cannot guess good tests, generate millions of random ones and watch what crashes. *Fuzzing* does exactly that: it feeds unusual inputs—empty strings, huge numbers, malformed files—to the program and monitors for crashes, hangs, or sanitiser warnings.

A naive fuzzer mutates blindly. A *coverage-guided* fuzzer (e.g., AFL, libFuzzer) instruments the program, remembers which mutations reached new branches, and breeds from those seeds. An *LLM-guided* fuzzer goes further: it learns which byte patterns are syntactically plausible (e.g., valid JSON, valid JavaScript) and mutates structure-aware, reaching deeper code faster.

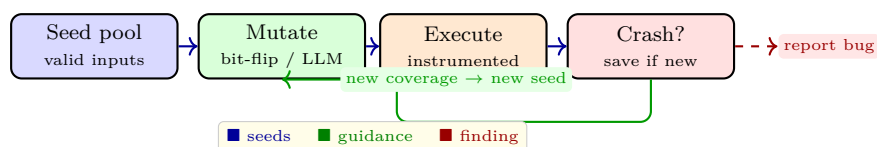


Figure 3.3: Coverage-guided fuzzing loop. Seeds that discover new branches are retained; LLM guidance makes mutations more likely to be well-formed and thus reach deeper logic.

Practical fuzzing needs three ingredients: a *harness* (how to call the target), a *mutator* (how to change inputs), and a *sanitiser* (AddressSanitizer, UBSan) that turns silent corruption into a visible crash. AI helps mainly with the mutator and harness generation.

3.4 The Oracle Problem and How to Sidestep It

If inputs are easy and oracles are hard, the bottleneck of automated testing is not generation but *judgement*. Three workarounds dominate.

Differential oracles: run two implementations of the same spec (e.g., two regex engines) on the same input; disagreement signals a bug in at least one. *Metamorphic relations*: specify how outputs should relate when inputs are transformed. For `sort`, shuffling the input should not change the output; for a compiler, adding dead code should not change the generated program. *Implicit oracles*: any crash, hang, or sanitiser error is a failure even without a golden output—the basis of fuzzing.

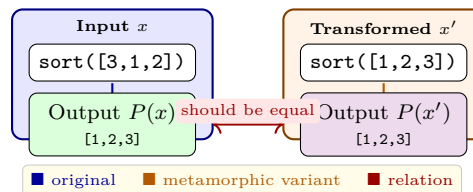


Figure 3.4: Metamorphic oracle: shuffling the input to `sort` should not change the sorted output. The relation checks the program against itself, no golden output needed.

Example 3.1 (Metamorphic relation for median). If x' is x with every element shifted by $+c$, then $\text{median}(x') = \text{median}(x) + c$. A generated test can check this relation on random x and c without knowing the true median in advance. Violations reveal bugs in handling of negatives or overflow.

No oracle is universal. The art is choosing a cheap partial oracle that catches the faults you fear most, and pairing it with a generator that explores where those faults hide.

Remark 3.1 (Testing as specification). A test suite is an executable partial specification. The oracle problem is therefore a specification problem: the more precisely you can state what “correct” means (as properties, relations, or a second implementation), the more you can automate checking. AI helps by inferring likely properties from code and comments, but human judgement remains the ground truth.

Beyond crashes: non-crash oracles for fuzzing. Modern fuzzers do not only look for crashes. They check for memory errors (sanitisers), performance regressions (timeouts), and differential discrepancies. The same loop in Fig. 3.3 applies, but the oracle box is pluggable—swap crash detection for a metamorphic check and the fuzzer searches for logic bugs instead.

3.5 Chapter Notes

EvoSuite (Fraser & Arcuri, 2011) remains the canonical search-based generator; recent LLM test-generation surveys compare it to Codex/GPT-based approaches. AFL and libFuzzer define coverage-guided fuzzing; recent

work (e.g., TitanFuzz, Fuzz4All) uses LLMs as structure-aware mutators. The oracle problem is surveyed by Barr et al. (2015); metamorphic testing was introduced by Chen et al. (1998) and is widely used for compilers, ML systems, and databases.

3.6 Exercises

- E3.1 **Coverage.** For the program in Fig. 3.1, write two tests achieving 100% line coverage but only 50% branch coverage, and two tests achieving 100% branch coverage. Explain why branch coverage subsumes line coverage but not vice versa, in one sentence.
- E3.2 **Oracle design.** For a function `reverse(s)` that reverses a string, give (a) an exact oracle, (b) a metamorphic relation, and (c) an implicit oracle. For each, state one bug it would catch and one it would miss.
- E3.3 **Generate–filter.** You generate 100 LLM tests; 60 compile, 40 increase line coverage, and 10 of those also increase mutation score. Propose a filtering pipeline that keeps the most valuable tests first. How would you avoid keeping 40 near-duplicate tests that all cover the same new line?
- E3.4 **Fuzzing feedback.** Explain why a blind random fuzzer struggles to reach the `else` branch of `if (x==0xdeadbeef) crash();`. How does coverage guidance help? How might an LLM mutator that has seen many hex constants help even more?
- E3.5 **Hands-on.** Prompt any public LLM with: “Write pytest tests for `def median(xs)` covering empty, duplicates, negatives, and large inputs.” Run the generated tests. Report one test that has a wrong oracle (asserts the buggy behaviour) and fix it. What does this reveal about trusting generated oracles?

Chapter 4

Automated Program Repair

“Fixing a bug is not just changing a line; it is arguing that the new line is more correct than the old one.” — Automated repair makes that argument executable: generate a candidate, show that it passes the tests, and invite human judgement on whether the argument holds.

From zero: no repair background assumed. We start from a failing test and the question “what would make it pass?”—how to find where the bug lives, how to propose a change, and how to check the change does not break something else. Every technique is shown as picture, then rule, then example.
--

Learning Outcomes

After this chapter you will be able to:

- (L1) define fault localisation, patch generation, and validation;
- (L2) trace the GenProg loop (evolutionary search over patches);
- (L3) explain template-based and learning-based repair (TBar, SequenceR);
- (L4) distinguish plausible from correct patches and overfitting;
- (L5) evaluate a repair tool with standard metrics and benchmarks.

4.1 What Is Automated Program Repair?

Intuition. You have a program P , a test suite T where at least one test fails, and you want a program P' that passes all of T and does not introduce new failures. The search space is enormous—every possible edit to P —so repair techniques differ in how they narrow it: heuristic search, fixed templates, or learned patterns from past fixes.

The standard pipeline has three stages. *Fault localisation* ranks suspicious lines (often with spectrum-based scoring: lines executed mostly by failing tests rank high). *Patch generation* proposes edits at those lines.

Validation runs T (and sometimes extra generated tests) to keep only *plausible* patches—those passing T . A plausible patch is not necessarily *correct*; it may overfit to the tests.

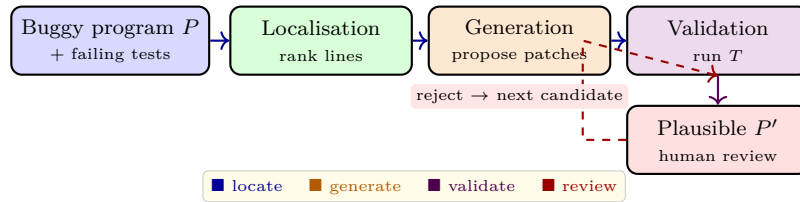


Figure 4.1: The APR pipeline. Localisation narrows *where*; generation proposes *what*; validation checks against T . Only plausible patches reach human review, and review decides correctness.

Definition 4.1 (Plausible vs. correct). A *plausible* patch passes all tests in T . A *correct* patch is plausible and also satisfies the intended specification (what the developer meant). Overfitting—passing T while violating the spec—is the central failure mode of APR.

Small example: `if (x > 0) y=1/x;` with test `x=0 should not crash`. The empty fix `if (true) y=0;` passes the test but is not correct; a correct fix guards the division.

Example 4.1 (Spectrum-based localisation). Suppose line L_1 is executed by 3 failing and 1 passing test, L_2 by 0 failing and 4 passing. Ochiai score $s(L) = \frac{\text{failed}(L)}{\sqrt{(\text{failed}(L) + \text{passed}(L)) \cdot \text{totalFailed}}}$ ranks L_1 much higher. The intuition is simple: suspiciousness grows with failing coverage and shrinks with passing coverage.

4.2 GenProg and Search-Based Repair

Intuition. If you do not know the right fix, try many small edits and keep those that get closer to passing. GenProg treats a program as a genome: *mutation* changes a line, *crossover* swaps edits between candidates, and *fitness* is the number of tests passed.

Concretely, GenProg starts from P , localises suspicious statements, and creates a population of variants by copying, deleting, or swapping statements drawn from elsewhere in the program (the *plastic surgery hypothesis*: the fix ingredient often exists nearby). Each variant is executed on T ; fitter variants breed the next generation. When a plausible patch appears, the search stops and the patch is minimised (redundant edits removed).

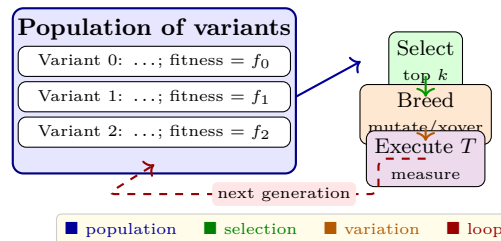


Figure 4.2: GenProg as evolutionary search. Fitness is tests passed; mutation and crossover propose nearby programs; selection keeps the fittest. The loop is a direct analogue of generate–execute–filter from testing.

GenProg was the first to fix real C bugs automatically (e.g., in `libtiff`, `php`) with no hand-written templates. Its strength is generality; its weakness is that random edits rarely produce a correct multi-line fix and the search is expensive.

Why plastic surgery works (and when it does not). Large codebases are repetitive: error handling, null checks, and boundary conditions appear many times in slightly different forms. Copying a nearby correct snippet is often enough for small bugs. It fails when the needed code does not already exist in the project (novel logic) or when the fix requires coordinated changes across files, which single-statement edits cannot express.

Example 4.2 (A GenProg-style fix). Buggy: `if (x != 0) y = 1/x; else y = 0;` with test `x=0 => y=0` passes but `x=0.0` still divides by zero due to typed comparison. A variant that copies a null-guard from nearby—`if (x==0) y=0; else y=1/x;`—is found by mutation and scores higher.

4.3 Learning Fix Patterns: From TBar to SequenceR

Search is general but blind; learning makes it sighted. *Template-based* repair (e.g., TBar) mines common fix templates from human patches—null-check insertion, bound adjustment, operator swap—and tries those first at suspicious locations. *Neural* repair (e.g., SequenceR, Recoder, AlphaRepair) trains a seq2seq or LLM to translate buggy code into fixed code, learning from millions of before–after pairs.

Listing 4.1: Template vs. learned fix: the same null dereference repaired two ways.

```

1  // Buggy Java: null dereference on line 3
2  String greet(String name) {
3      int len = name.length(); // NPE if name == null
4      return "Hello " + name;
5  }
6  // Template fix (TBar: Insert null check)
7  String greet(String name) {
8      if (name == null) return "Hello guest";
9      int len = name.length();
10     return "Hello " + name;
11 }
12 // Neural fix might instead produce:
13 // if (name == null) name = "guest"; // learned variant

```

The workflow for neural repair: input is the buggy snippet plus surrounding context and an error message; the model outputs a patch (often as a diff). Decoding is done with beam search; top- k patches are validated against T . Training data is mined from GitHub commits whose messages contain “fix” and whose diff is small. A crucial detail is *context*: the same buggy line may need different fixes depending on imports, types, and nearby methods, so modern models include a window of surrounding code, not just the single line.

Bias in learned fixes. Models reproduce what they have seen most: if training data contains many null-check insertions, the model will propose null checks even when a bound check is correct. Diversity in training fixes and explicit ranking by test outcome (not just likelihood) help, but the bias remains. This is why hybrid systems that try templates first and fall back to neural sampling often outperform pure neural approaches.

Listing 4.2: Python: off-by-one fix as a one-token change. A model that has seen many such pairs learns the pattern.

```

1  # Buggy
2  def sum_range(arr, lo, hi):

```

```

3   s = 0
4   for i in range(lo, hi): # bug: should be hi+1 if inclusive
5       s += arr[i]
6   return s
7
8   # Fixed (inclusive upper bound)
9   def sum_range(arr, lo, hi):
10      s = 0
11      for i in range(lo, hi+1):
12          s += arr[i]
13      return s
14  # Test that distinguishes: sum_range([1,2,3],0,2) should be 6

```

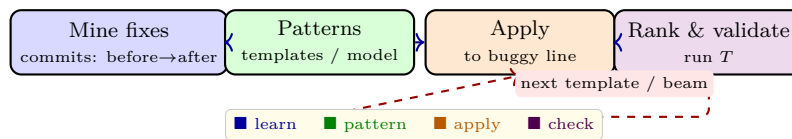


Figure 4.3: Learning fix patterns. Past fixes are mined into templates or a neural model; at repair time the model proposes patches at localised lines and they are ranked by validation on T .

Learning helps with common, local fixes (one–two lines, familiar APIs) and struggles with novel, multi-file, or specification-heavy bugs. The best recent systems hybridise: templates handle known patterns quickly, neural models handle the long tail, and search fills gaps when both miss.

Context is critical. The same buggy line `name.length()` needs different fixes depending on whether `name` is allowed to be null (add guard) or should never be null (fix the caller). Modern neural repair models therefore feed the model not just the buggy line but a window of surrounding method, the failing test message, and sometimes the stack trace. Without that context, even a perfect learner guesses.

Example 4.3 (How much context matters). With only the line `int len = name.length();`, a model cannot know whether the intended fix is `if (name==null) return 0;` or `if (name==null) name=""`; With the caller `greet(null)` and test `assertEquals("Hello guest", greet(null))` in context, the second is clearly preferred. This is why APR is increasingly framed as *retrieval-augmented generation*: fetch relevant tests and docs, then generate.

A useful analogy: template repair is a phrasebook (fast for known phrases), neural repair is a fluent speaker (general but occasionally wrong), and search is trial-and-error (slow but needs no prior knowledge). Good pipelines try them in that order.

In practice, teams combine all three: mine templates from the project’s own history, fine-tune a neural model on that history, and fall back to search when both fail. The shared bottleneck remains validation—without strong tests, every approach overfits.

4.4 Evaluation, Overfitting, and Limits

A repair tool is judged by *plausible rate*, *correct rate* (human-judged), and *precision* (correct/plausible). Benchmarks like *Defects4J* (Java, 800+ real bugs), *ManyBugs* (C), and *QuixBugs* provide P and T ; leakage and weak

T are constant threats. Tools often report “ X bugs fixed” without distinguishing plausible from correct—always check which number is quoted.

The central pitfall is *test overfitting*: a patch that deletes functionality can pass a weak suite (e.g., removing the feature that has the bug). Mitigations include augmenting T with generated tests (as in Ch. 3), checking against a held-out oracle when available, and requiring minimal edits. In practice, combining a weak APR patch with one human-written regression test often turns a plausible patch into a correct one.

Definition 4.2 (Overfitting in repair). A patch *overfits* T if it passes T but fails outside it—it fits the tests, not the spec. Stronger validation (more tests, not more search) is the primary defence. Two common overfit patterns are *test-suite overfitting* (the patch hard-codes the expected output for a specific test input) and *incomplete fixing* (it repairs one failing test while breaking an untested behaviour).

Remark 4.1 (What repair cannot do). APR is best at small, local, pattern-following fixes where tests are strong—typically one to three lines in a single method. It is not a substitute for understanding the specification. For a novel feature or a design flaw, the correct action is to clarify the spec and write the code deliberately—the assistant proposes, the engineer decides, as in every chapter.

4.5 Chapter Notes

GenProg (Le Goues et al., 2012) founded search-based repair; Defects4J (Just et al., 2014) standardised evaluation. TBar (Liu et al., 2019) systematised template mining; SequenceR (Chen et al., 2019), Recoder, and AlphaRepair represent the neural line. The overfitting analysis of Qi et al. and Smith et al. is essential reading on why plausible $\neq$ correct and how augmented testing helps. Recent surveys (Goues et al., 2022; Zhang et al., 2023) map the full design space of localisation, generation, and validation choices.

Remark 4.2 (Practical tip). When triaging a tool’s patch, ask: does it generalise beyond the failing test? A quick check is to write one more test that the fix should satisfy but the buggy code did not (e.g., `greet("")` or `greet("A very long name")`). If the patch fails that new test, it likely overfits.

4.6 Exercises

E4.1 Pipeline. For a buggy `median` that returns wrong output on even-length lists, walk through Fig. 4.1: which lines would spectrum-based localisation rank highest? What templates would TBar try first? How would validation decide?

E4.2 GenProg fitness. A suite has 10 tests, 3 failing on P . Variant A passes 8 tests, variant B passes 7 but fixes one previously failing test while breaking a previously passing one. Which has higher fitness? Which would you prefer to breed from? Propose a fitness that rewards fixing failing tests more than preserving passing ones.

E4.3 Template vs. neural. Give a bug where a template is likely to succeed and one where a neural model might succeed while templates fail. Justify each in one sentence by referring to fix locality and pattern frequency.

- E4.4 **Overfitting.** A repair deletes the buggy method body and returns a constant that happens to pass all existing tests. Explain why this is plausible but not correct. Propose two concrete additions to T that would reject it: one coverage-guided test and one metamorphic test.
- E4.5 **Hands-on.** Take the Java `greet` bug in Listing 4.1. Prompt any LLM with the buggy method plus the failing test `assertEquals("Hello guest", greet(null))` and ask for a minimal fix. Validate the patch by running the test. Is the patch plausible? Is it correct for all callers? Argue in two sentences.

Chapter 5

Code Quality and Intelligent Review

If software is a bridge, code quality is its structural integrity — and AI is the inspector that never blinks.

From zero: no quality background assumed. We start from “what makes code good?” and build up metrics, inspection, automated review, and ML-enhanced analysis step by step. Every term is defined before use; pictures precede formulas.

Learning Outcomes. After this chapter you will be able to (1) define and compute classic quality metrics; (2) contrast static analysis, dynamic analysis, and testing; (3) explain how ML augments static analysis; (4) describe AI-assisted code review pipelines; (5) evaluate trade-offs of automated quality gates.

5.1 Intuition: What Is “Good” Code?

A program can be *correct* and still be terrible: a 500-line function with nested `ifs`, cryptic names, and no tests will break the moment someone else touches it. Quality captures how well code supports **change**. Three lenses help.

- **External quality** — does it do what users need? Correctness, reliability, performance, security.
- **Internal quality** — can developers understand and modify it? Readability, modularity, complexity, testability.
- **Process quality** — can the team deliver it sustainably? Review thoroughness, build health, defect escape rate.

AI for Software Engineering (AI4SE) targets all three, but this chapter focuses on internal quality where AI has made the fastest progress: measuring it, analysing it, and reviewing it before humans spend time.

5.2 Measuring Quality: Metrics that Matter

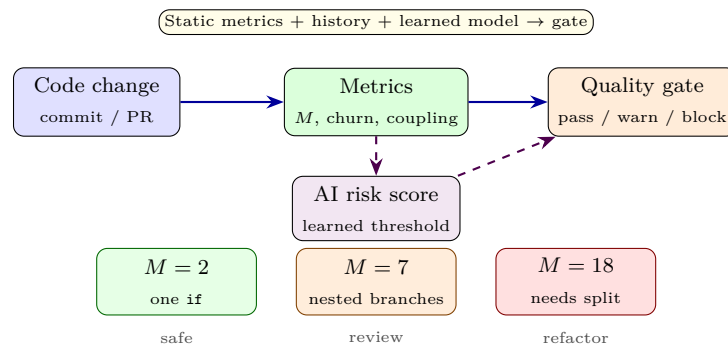
Metrics turn vague worry (“this file feels risky”) into numbers. The oldest still dominates.

Definition 5.1 (Cyclomatic Complexity — McCabe 1976). For a control-flow graph with E edges, N nodes, and P connected components,

$$M = E - N + 2P.$$

Equivalently, M is one plus the number of decision points (`if`, `while`, `case`, `&&`, `||`). A straight-line function has $M = 1$; each branch adds one independent path.

Teams often gate on $M \leq 10$ –15 per function; above 20, testing every path becomes impractical. Other staples include **coupling** (how many modules depend on you), **cohesion** (do a module’s members belong together), **code churn** (lines added/removed recently — churn predicts bugs), and **maintainability index** (a composite of M , Halstead volume, and lines of code).



Metrics alone do not fix code; they *flag* where to look. The next two sections give the eyes and the judgement.

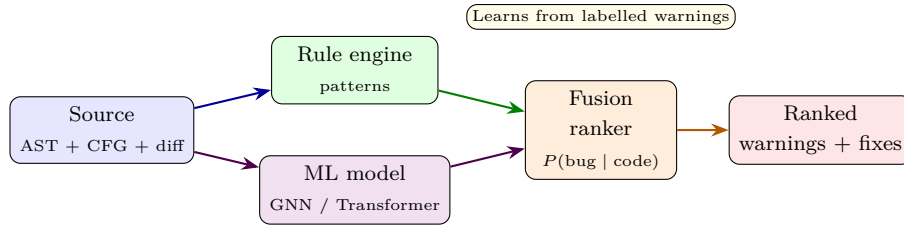
5.3 Static Analysis: Traditional to Learning-Augmented

Static analysis inspects code without running it: parsing, building an abstract syntax tree (AST), a control-flow graph (CFG), or a data-flow lattice, then checking rules.

Classic analysers are **rule-based**: “never compare `==` on floats,” “close every opened file,” “no SQL string concatenation.” Tools like ESLint, SpotBugs, SonarQube, and Infer encode thousands of such patterns. They are precise on what they encode and blind to everything else, producing many false positives on idiomatic code.

ML augments static analysis in three ways:

1. **Learned detectors.** Train a classifier on AST/CFG embeddings to flag bug-prone patterns the rule writer never anticipated. Graph neural networks (GNNs) over CFGs and Transformers over token streams are common encoders.
2. **Ranking and triage.** A model scores each warning by likelihood of being a true bug, suppressing noise. Developers see the top- k most actionable findings.
3. **Automated repair hints.** Given a flagged snippet, a generative model proposes a fix (“add null check at line 42”) trained on past human fixes.

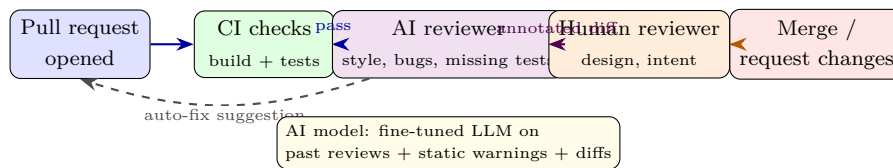


A practical hybrid: run the rule engine first (sound, fast, explainable), then let the ML ranker re-order and annotate. Teams that tried pure ML quickly learned that developers distrust black-box flags with no rule-like explanation; hybrids keep the explanatory rule trace.

5.4 AI-Assisted Code Review

Code review is where judgement scales least: a senior engineer reads a diff, imagines executions, and comments. AI reviewers do not replace that, but they *pre-review* — catching style, common bugs, missing tests, and even design smells before a human is asked.

A modern review pipeline looks like this:



What AI reviewers do well: surface duplicates, flag missing edge-case tests, suggest clearer names, detect API misuse, and check that new code follows project conventions learned from history. What they still need humans for: Is this the *right* abstraction? Does the change match the requirement's intent? Is the added dependency worth it?

Listing 5.1: Rule check plus a toy ML ranker for warning triage.

Example 5.1 (Linter + ML ranker in Python).

```

1 import ast, re
2
3 # --- Rule-based: flag bare except and high-complexity functions ---
4 def rule_warnings(source: str):
5     tree = ast.parse(source)
6     warns = []
7     for node in ast.walk(tree):
8         if isinstance(node, ast.Try):
9             for h in node.handlers:
10                 if h.type is None: # bare except:
11                     warns.append((node.lineno, "bare-except",
12                                   "Avoid bare except: catches SystemExit/KeyboardInterrupt"
13                                   ""))
14
15     # naive complexity: count branching keywords
16     complexity = len(re.findall(r'\b(if|for|while|and|or|except)\b', source))
17     if complexity > 12:
18         warns.append((1, "high-complexity",
19                       "Consider simplifying branching logic: too many keywords."))
20
21 return warns
  
```

```

17         f"Complexity ~{complexity}: consider splitting function"))
18     return warns
19
20 # --- ML-style ranker (toy: score by churn + complexity) ---
21 def rank_warnings(warns, churn_lines=0, complexity=0):
22     scored = []
23     for line, kind, msg in warns:
24         # learned weights would come from a trained model; here heuristic
25         score = 0.7 * (kind == "bare-except") + 0.3 * (complexity / 20.0) + 0.1 * (
26             churn_lines > 50)
27         scored.append((score, line, kind, msg))
28     return sorted(scored, reverse=True) # highest risk first
29
30 src = "try:\n    x = int(input())\nexcept:\n    pass\n"
31 for s, l, k, m in rank_warnings(rule_warnings(src), churn_lines=80, complexity=14):
32     print(f"[{s:.2f}] L{l} {k}: {m}")

```

Evaluation of review AI uses *precision@k* (are the top- k flags real?), *developer acceptance rate* (did the author apply the suggestion?), and *time saved* (does pre-review shorten human review latency?). The best systems optimise acceptance, not just accuracy — a correct but pedantic comment that developers dismiss helps no one.

5.5 Quality Gates and Developer Experience

A **quality gate** is an automated check that blocks or warns on merge when thresholds are breached: coverage drops below 80%, new $M > 15$ functions appear, or AI risk score exceeds a learned cutoff. Gates should be *fast* (seconds, not minutes), *explainable* (link to the rule or example that triggered), and *tunable* (teams set their own thresholds).

The danger is alert fatigue: too many gates and developers route around them. Good practice is to (i) start with warnings, promote to blocks only after false-positive rate is low, (ii) show the fix, not just the flag, and (iii) measure whether the gate actually reduces escaped defects in the next release.

5.6 Worked Example: From Metric to Action

Consider a Python function that grew organically:

Listing 5.2: A function whose complexity is easy to misjudge.

```

1 def process(order, user, cfg):
2     if not order: return None
3     if user.is_admin or user.id == order.owner:
4         if cfg.get("fast"):
5             if order.priority > 5 and order.value > 1000:
6                 return expedite(order)
7             else:
8                 return queue(order)
9     else:
10        validate(order)

```

```

11         return store(order)
12     else:
13         raise PermissionError

```

Decision points: `if not order` (1), or in the admin check (2), outer `if/else` (3), `if cfg fast` (4), and with two comparators (5,6). So $M \approx 7$ by counting, or $E = 14, N = 9, P = 1 \Rightarrow M = 14 - 9 + 2 = 7$. That is still reviewable, but adding one more nested condition would push it past 10. The fix is mechanical: extract `can_access(user,order)` and `route_fast(order,cfg)` as helpers. Each helper drops to $M \leq 3$, becomes unit-testable, and the ranker score falls because churn on a smaller function is less risky.

5.7 Common Pitfalls

- **Goodhart’s law on metrics.** If you gate strictly on $M \leq 10$, developers game it by splitting functions without improving design. Pair metric gates with review of cohesion and naming.
- **Ignoring false positives.** A ranker at precision 0.3 wastes two-thirds of review time. Calibrate on your repo; do not copy thresholds from a paper’s open-source corpus.
- **ML without provenance.** A black-box “high risk” flag without a linked rule or example is dismissed. Always attach evidence: which pattern, which similar past bug, which file history.
- **Autofix without tests.** Auto-applying an LLM fix that passes lint but breaks a hidden invariant is worse than no fix. Every auto-fix must run the relevant tests before surfacing.

5.8 Chapter Summary

Quality is changeability. Metrics like $M = E - N + 2P$ map structure to testability; static analysis checks rules without execution; ML adds learned detection, ranking, and repair hints; and AI reviewers pre-screen diffs before humans weigh in. The future is hybrid: deterministic rules for soundness, learned models for coverage and prioritisation, and humans for design judgement.

5.9 Exercises

- E5.1 **Complexity by hand.** Draw the CFG for `if a and b: x=1 elif c: x=2 else: x=3` and compute $M = E - N + 2P$. Count decision points and verify both methods agree. At what M would you recommend splitting the function?
- E5.2 **Rule vs. learned.** Give one bug a rule-based linter catches reliably and one it cannot, then sketch how a GNN over the CFG could catch the second. What training data would you need for the GNN?
- E5.3 **Review pipeline.** Trace a PR that adds a function with $M = 19$ and a bare `except` through the pipeline in Fig. 3. Which stage flags which issue, and what happens if the AI reviewer is bypassed?
- E5.4 **Ranking metric.** You have 200 warnings, 30 are true bugs. Your ranker puts 12 true bugs in the top

20. Compute precision@20, recall@20, and F_1 @20. Is this good enough to auto-block merges? Justify.

E5.5 **Gate design.** Design a quality gate for a team that currently has 200 SonarQube warnings on `main`. Propose thresholds (M , coverage, AI risk score), a rollout plan (warn $\rightarrow$ block), and how you would measure whether escaped defects fall.

Chapter 6

Natural Language for Requirements

Requirements are wishes written in human language — beautiful, messy, and famously misunderstood. NLP helps us read them faithfully.

From zero: no NLP or RE background assumed. We start from “what is a requirement?” and build up ambiguity, classification, and traceability before any model or formula. Pictures first, then pipelines.

Learning Outcomes. After this chapter you will be able to (1) distinguish functional and non-functional requirements; (2) name and detect ambiguity types; (3) apply NLP steps to requirements text; (4) formulate traceability as link recovery; (5) evaluate an RE-NLP tool with precision/recall.

6.1 Intuition: Why Requirements Are Hard

Every project starts with language,

“The system shall allow premium users to export reports quickly.”

What does *quickly* mean? Two seconds? Ten? Is “export” a download or an email? Who counts as *premium*? One sentence smuggles three ambiguities.

Requirements Engineering (RE) is the discipline of turning such wishes into verifiable statements. It has two halves: **elicitation** (finding out what stakeholders want) and **specification** (writing it so builders and testers agree on done). Between them lies natural language — expressive enough for any wish, and vague enough to hide every misunderstanding until it becomes a bug.

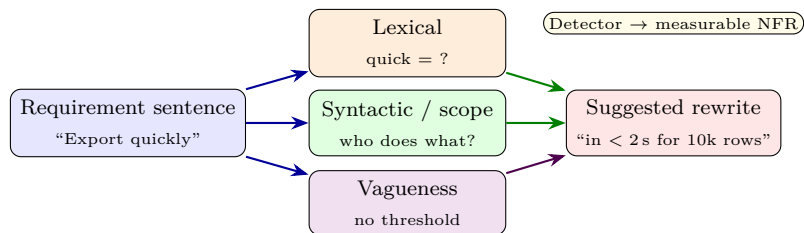
Classically we distinguish **functional requirements** (FR: what the system *does* — “export as CSV”) from **non-functional requirements** (NFR: how well it does it — “in under 2s for 10k rows,” security, usability, availability). NFRs are where language is vaguest and AI helps most.

6.2 Ambiguity: A Taxonomy You Can Act On

Not all ambiguity is the same; different types need different detectors.

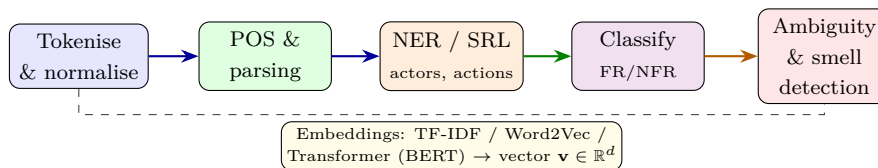
- **Lexical** — a word has multiple senses. “*The system shall process the order*” (purchase order? execution order?).
- **Syntactic** — grammar admits two parses. “*Notify managers of users who failed login*” — who failed?
- **Semantic / scope** — quantifiers and conditions are unclear. “*All premium reports shall be exported*” (all reports belonging to premium users, or a report type called premium?).
- **Pragmatic / vagueness** — adjectives without thresholds. “*Fast,*” “*user-friendly,*” “*sufficient.*”
- **Referential** — pronouns or definite descriptions lack a clear antecedent. “*It shall be encrypted*” (what is *it*?).

A classic remedy is boilerplates and controlled language (EARS: *When <trigger>, the system shall <action>*). NLP detectors can flag deviations from such templates and suggest a rewrite with explicit thresholds and actors.



6.3 NLP Pipeline for Requirements

Turning prose into actionable structure needs a pipeline. Inputs are raw requirement documents, user stories, app reviews, or interview transcripts; output is classified, structured, and linked artefacts.



Key steps in plain language:

1. **Tokenisation & normalisation** — split text into tokens, lemmatise (*exporting*→*export*), handle domain jargon.
2. **POS tagging & parsing** — label nouns/verbs and recover grammatical structure; features for syntactic-ambiguity detection.
3. **Named-entity / semantic-role labelling** — find *who* acts on *what* (actor, action, object, condition).

4. **Vectorisation** — map each requirement to a vector $\mathbf{v}$: sparse TF-IDF for baselines, dense BERT embeddings for state of the art. Similarity is then $\cos(\mathbf{v}_i, \mathbf{v}_j) = \mathbf{v}_i^\top \mathbf{v}_j / (\|\mathbf{v}_i\| \|\mathbf{v}_j\|)$.
5. **Classification & detection** — FR vs. NFR vs. sub-classes (security, performance, usability), and binary flag *ambiguous?* plus span highlighting.

Listing 6.1: TF-IDF + logistic regression for FR/NFR; rule-assisted vagueness check.

Example 6.1 (Lightweight RE classifier and vagueness spotter).

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.linear_model import LogisticRegression
3
4 reqs = [
5     "System shall export report as CSV.",          # FR
6     "Export shall complete in under 2 seconds.",    # NFR-performance
7     "System shall allow premium users to export quickly.", # vague NFR
8     "User can filter reports by date range.",        # FR
9     "Data at rest shall be AES-256 encrypted.",      # NFR-security
10 ]
11 labels = ["FR", "NFR", "NFR", "FR", "NFR"] # toy supervision
12
13 vec = TfidfVectorizer(ngram_range=(1,2), stop_words="english")
14 X = vec.fit_transform(reqs)
15 clf = LogisticRegression().fit(X, labels)
16 print(clf.predict(vec.transform(["Export shall be fast."]))) # likely NFR
17
18 # Rule-assisted vagueness dictionary (complements ML)
19 VAGUE = {"quickly", "fast", "user-friendly", "sufficient", "efficient", "robust"}
20 def flag_vague(sent: str):
21     toks = sent.lower().split()
22     hits = [t.strip(".") for t in toks if t.strip(".") in VAGUE]
23     return hits # e.g. ["quickly"] -> suggest numeric threshold
24
25 print("vague terms:", flag_vague(reqs[2]))
26 # cosine similarity between two requirements (for traceability preview)
27 from sklearn.metrics.pairwise import cosine_similarity
28 sims = cosine_similarity(vec.transform([reqs[0]]), vec.transform([reqs[3]]))
29 print(f"similarity FR-FR: {sims[0,0]:.2f}")

```

Transformers (BERT, RoBERTa) fine-tuned on labelled RE corpora (e.g., PROMISE NFR dataset) beat TF-IDF on FR/NFR classification by 5–15 pp, but they need more data and calibration. A practical compromise is *hybrid*: rules for high-precision vagueness lists, ML for broader classification, and humans to adjudicate flagged cases.

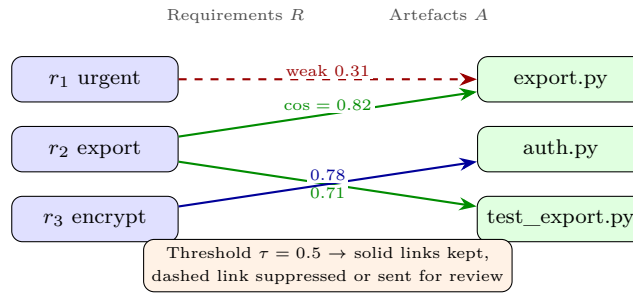
6.4 Traceability: Linking Wishes to Code and Tests

A **trace link** connects a requirement to the artefact that implements or verifies it: design, code file, or test case. The full set of links is the **traceability matrix** $T \in \{0,1\}^{|R| \times |A|}$ where R are requirements and A artefacts. Maintained well, T answers “if we change requirement r_i , what breaks?” and “is every requirement covered by a test?”

Manual tracing is tedious and rots as code evolves. **Automated traceability** is an information-retrieval and classification problem: given (r_i, a_j) , predict $P(\text{link} \mid r_i, a_j)$.

Approaches in order of power:

- **Textual similarity (IR)** — compute $\cos(\mathbf{v}_{r_i}, \mathbf{v}_{a_j})$ using TF-IDF or embeddings of requirement plus code comments/identifiers. Threshold to propose links. Simple, surprisingly effective.
- **Supervised classification** — train on confirmed links (positive) and sampled non-links (negative); features include textual similarity, co-change history, and file proximity.
- **Deep bi-encoders** — encode requirement and artefact with separate Transformers, score by dot product; fine-tuned on project links with contrastive loss.
- **LLM-assisted** — prompt an LLM with requirement + candidate file excerpt and ask “does this file implement r_i ?” with chain-of-thought; useful when code identifiers are cryptic.



Evaluation mirrors information retrieval: for a requirement r_i , let L_i be true linked artefacts and $\hat{L}_i(\tau)$ those predicted above threshold τ ,

$$\text{Precision} = \frac{|L_i \cap \hat{L}_i|}{|\hat{L}_i|}, \quad \text{Recall} = \frac{|L_i \cap \hat{L}_i|}{|L_i|}, \quad F_1 = \frac{2PR}{P + R}.$$

In practice recall matters more than precision for traceability (missing a link hides impact), so teams favour thresholds that keep $\text{recall} \geq 0.8$ and let humans prune false positives. Maintain links incrementally: re-score only changed artefacts on each commit.

6.5 Putting It Together: An RE Assistant

A lightweight RE assistant chains the pieces: on every commit or spec edit, it (i) classifies new sentences as FR/NFR, (ii) flags ambiguous spans with suggested rewrites, and (iii) proposes trace links to code/tests for a human to confirm. The human stays in the loop — the tool reduces the search space from hundreds of candidates to a shortlist, and it explains each suggestion (highlighted vague term, similarity score, or co-change evidence).

6.6 Worked Example: End-to-End on a Sprint

A team receives five new user stories. We run the pipeline:

1. Classifier labels: two FR (“filter by date,” “export CSV”), three NFR (“encrypt at rest,” “complete in < 2 s,” “support SSO”).
2. Ambiguity detector flags “quickly” in an older story and suggests “in < 2 s for 10k rows on p95” — the product owner confirms the threshold, removing a future performance bug.
3. Traceability proposes links: FR “export CSV” $\rightarrow$ `export.py` (cos 0.82) and `test_export.py` (0.71); NFR “encrypt” $\rightarrow$ `auth.py` (0.78). The weak 0.31 link to `export.py` from an unrelated reliability story is suppressed for manual review.

A second listing verifies the links with a readable helper:

Listing 6.2: Thresholding trace links and reporting $P/R/F_1$.

```

1 def prf(true_links, pred_links):
2     tp = len(true_links & pred_links)
3     prec = tp / len(pred_links) if pred_links else 0.0
4     rec = tp / len(true_links) if true_links else 0.0
5     f1 = 2*prec*rec/(prec+rec) if (prec+rec) else 0.0
6     return prec, rec, f1
7
8 true = {("r2", "export.py"), ("r2", "test_export.py"), ("r3", "auth.py")}
9 pred_05 = {("r2", "export.py"), ("r2", "test_export.py"), ("r3", "auth.py")} # tau=0.5
10 pred_08 = {("r2", "export.py"), ("r3", "auth.py")} # tau=0.8
11 print("tau=0.5", prf(true, pred_05)) # (1.0, 1.0, 1.0) on this toy set
12 print("tau=0.8", prf(true, pred_08)) # (1.0, 0.67, 0.80) -- recall drops

```

6.7 Common Pitfalls

- **Evaluating on shuffled data.** Requirements evolve; random splits leak future vocabulary into training. Split temporally (train on past releases, test on next).
- **Ignoring class imbalance.** NFR sub-classes (e.g., security) are rare; accuracy looks high while the rare class is never predicted. Report per-class F_1 and macro- F_1 .
- **Over-trusting similarity.** High cosine similarity can come from shared boilerplate (“the system shall”) rather than semantic overlap. Down-weight boilerplate via IDF or remove it before embedding.
- **Trace decay.** Links rot when files move. Re-score incrementally on each rename or refactor and flag orphaned requirements with no surviving artefact.

6.8 Chapter Summary

Requirements live in language, and language is ambiguous. By naming ambiguity types, applying a standard NLP pipeline (tokenise $\rightarrow$ parse $\rightarrow$ embed $\rightarrow$ classify), and framing traceability as ranked link recovery evaluated with precision/recall/ F_1 , we turn subjective reading into measurable engineering. The best results are hybrid: rules where precision matters, learned embeddings where coverage matters, and human confirmation always.

6.9 Exercises

- E6.1 **Classify and rewrite.** Label each as FR, NFR, or ambiguous NFR and rewrite the vague one measurably:
 (a) “System shall support single sign-on via SAML.” (b) “Dashboard shall load quickly.” (c) “Logs shall be retained for audit purposes.”
- E6.2 **Ambiguity types.** For “Notify admins of users who exceeded quota,” identify which ambiguity type(s) apply and give two distinct readings. Propose an EARS-style sentence that removes the ambiguity.
- E6.3 **Embeddings by hand.** With TF-IDF vectors $\mathbf{v}_1 = [0.7, 0.2, 0]$, $\mathbf{v}_2 = [0.6, 0.3, 0.1]$, $\mathbf{v}_3 = [0, 0, 0.9]$ for three requirements, compute all pairwise cosine similarities. Which pair should be linked by a naïve $\tau = 0.5$ threshold, and is that sensible?
- E6.4 **Traceability metrics.** Requirement r truly links to $\{a_1, a_2, a_3\}$. Your tool predicts $\{a_1, a_2, a_4\}$ at $\tau = 0.6$ and $\{a_1, a_4\}$ at $\tau = 0.8$. Compute P, R, F_1 at both thresholds. Which threshold would you ship, and why?
- E6.5 **Mini pipeline.** Extend Listing 6.1 to add an NFR sub-classifier (security vs. performance vs. usability) on a toy dataset of 12 sentences. Report accuracy with 3-fold cross-validation and list the top vague terms flagged but misclassified.

Chapter 7

AI in DevOps: CI/CD, Defects, and Estimation

DevOps closes the loop from commit to customer; AI makes the loop faster, safer, and more predictable.

From zero: no DevOps background assumed. We build from “what is CI/CD?” through pipelines, feedback loops, and measurement before any model appears. Every formula is motivated by a picture.

Learning Outcomes. After this chapter you will be able to (1) sketch a CI/CD pipeline and its quality gates; (2) explain where AI plugs into DevOps; (3) formulate defect prediction as supervised learning; (4) contrast algorithmic and ML-based effort estimation; (5) measure DevOps impact with DORA-style metrics.

7.1 Intuition: The DevOps Loop

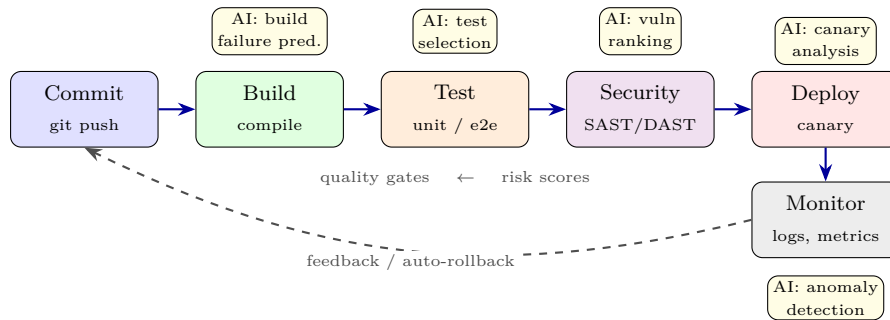
Traditional development threw code “over the wall” to operations. **DevOps** tears down the wall: one cross-functional team owns code, build, test, deploy, monitor, and learn — as a single loop. Two practices automate the loop:

- **Continuous Integration (CI)** — every commit triggers an automated pipeline: compile, unit-test, lint, security scan, and package. Failures are reported in minutes, not days.
- **Continuous Delivery/Deployment (CD)** — every green build is automatically staged, and (with deployment) released to users behind feature flags and canaries.

Lead time shrinks from weeks to hours. The cost is complexity: more automation means more logs, more signals, and more decisions about *what* to test, *where* bugs lurk, and *how long* work will take. Those decisions are where AI earns its keep.

7.2 AI Across the Pipeline

Think of the pipeline as an assembly line with inspectors. AI inspectors can stand at every station:



Concrete AI insertions that teams actually ship:

1. **Build-failure prediction** — before running the full pipeline, predict $P(\text{fail} \mid \text{diff})$ from diff size, churn, author history, and files touched; skip or prioritise.
2. **Test selection & prioritisation** — run the 5% of tests most likely to fail first (based on dependency graph + recent failures), so developers get signal in seconds.
3. **Log anomaly detection** — learn normal log templates (Drain, Spell) then flag rare templates or sequences that precede incidents.
4. **Canary analysis** — compare canary vs. baseline on latency/error metrics with statistical or ML-based change-point detection; auto-roll back on regression.

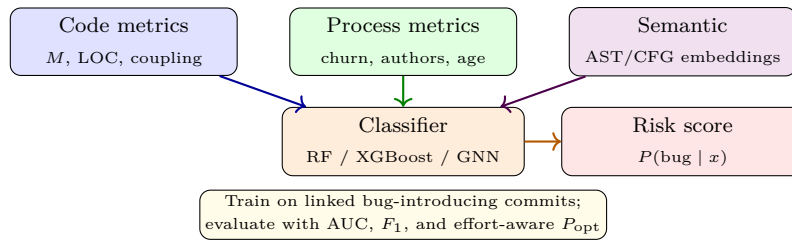
All four share a pattern: learn from history, score the present, and act (reorder, skip, or block) with an explainable reason.

7.3 Defect Prediction: Will This Change Break?

Defect prediction asks: given a file, commit, or pull request, what is the probability it introduces a bug? Early work predicted at *release* granularity (“which files will have post-release defects?”); modern work predicts at *commit* or *just-in-time* granularity, which is far more actionable.

Formally, for each instance x_i (a file or commit) with label $y_i \in \{0, 1\}$ ($1 = \text{buggy}$), learn $f : \mathcal{X} \rightarrow [0, 1]$ where $f(x_i) \approx P(y_i = 1 \mid x_i)$. Training data come from linking bug-fix commits back to the commits that introduced the bug (SZZ algorithm and variants — noisy but standard).

Features fall into three families:



Process metrics often beat code metrics alone: a file with high churn touched by many authors is riskier than a complex but stable file. Semantic embeddings (CodeBERT, GraphCodeBERT over AST/CFG) add further gains, especially for cross-project prediction where raw metric distributions shift.

Handling imbalance matters: only 10–20% of commits are buggy, so train with class weighting, SMOTE, or focal loss, and report **AUC** and **effort-aware** measures (P_{opt} , AUCEC) that ask “how many bugs do we find if we inspect the top 20% riskiest code?” rather than raw accuracy.

Example 7.1 Listing 7.1: JIT defect prediction with process + code features and XGBoost. (Just-in-time defect prediction in Python).

```

1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.metrics import roc_auc_score, classification_report
4 from xgboost import XGBClassifier
5
6 # Toy dataset: each row is a commit; label 1 = introduced a bug (via SZZ)
7 data = pd.DataFrame({
8     "churn": [12, 180, 5, 220, 30, 90, 15, 300, 8, 150],
9     "num_authors": [1, 4, 1, 5, 2, 3, 1, 6, 1, 3],
10    "cyclomatic": [3, 18, 2, 22, 5, 12, 4, 25, 2, 14],
11    "file_age_days": [200, 30, 400, 15, 180, 60, 300, 10, 500, 45],
12    "buggy": [0, 1, 0, 1, 0, 1, 0, 1, 0, 1],
13 })
14 X = data[["churn", "num_authors", "cyclomatic", "file_age_days"]]
15 y = data["buggy"]
16 Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
17
18 clf = XGBClassifier(use_label_encoder=False, eval_metric="logloss",
19                     scale_pos_weight=1.2, max_depth=3, n_estimators=80)
20 clf.fit(Xtr, ytr)
21 proba = clf.predict_proba(Xte)[:, 1]
22 print(f"AUC: {roc_auc_score(yte, proba):.2f}")
23 print(classification_report(yte, (proba > 0.5).astype(int)))
24 # Actionable use: rank PRs by proba, inspect top-k
25 ranked = sorted(zip(Xte.index, proba), key=lambda x: -x[1])
26 print("Inspect first:", ranked[:2])
  
```

Deployment tip: surface the score in the PR as “risk: high (churn 220, $M=22$, 5 authors)” and offer to auto-add reviewers or require extra tests — developers accept guidance that explains *why*.

7.4 Effort Estimation: How Long Will It Take?

Estimation is notorious: Brooks’s law, optimism bias, and shifting scope conspire against it. Two traditions meet here.

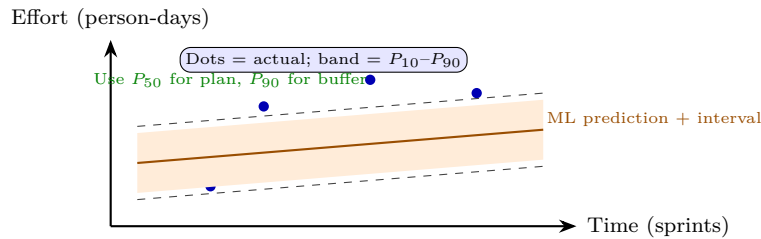
Algorithmic models encode expert judgement as formulas. The classic is COCOMO (Boehm, 1981):

$$\text{Effort (person-months)} = a \cdot (\text{KLOC})^b \cdot \prod_i \text{EM}_i,$$

where a, b depend on project class and EM_i are effort multipliers (required reliability, analyst capability, tool use, ...). COCOMO II refines this with scale factors (precedence, team cohesion) and reuse. These models are interpretable but need careful calibration per organisation.

ML-based estimation learns from history: given features (story points, function points, team velocity, textual description embeddings, dependency graph), regress to actual effort. Modern variants fine-tune language models on issue text + labels (hours or story points), or use analogy-based reasoning (find k nearest past issues and average their effort).

Evaluation must respect time: train on past sprints, test on future ones (no shuffling). Report **MAE**, **MMRE** $= \frac{1}{n} \sum |(\hat{y}_i - y_i)/y_i|$, and **PRED(25)** = fraction within 25% of true effort. No model beats seasoned judgement by a huge margin; the win is consistency and calibration — ML estimates come with a predicted interval, so planning can use P_{50} and P_{90} rather than a point guess.



7.5 Worked Example: Prioritising Tests with History

Suppose nightly CI runs 2000 tests but developers need feedback in under 3 min. Historical data show that 15 tests fail on 40% of buggy PRs. A simple AI prioritiser scores each test t by

$$s(t) = \alpha \cdot \text{fail_rate}(t) + \beta \cdot \text{cochange}(t, \Delta) + \gamma \cdot \text{distance}(t, \Delta),$$

where cochange is how often t and changed files Δ were modified together and distance is graph distance in the dependency graph. Sorting by $s(t)$ and running the top 5% first catches 70% of failures within the first minute. A second listing adds confidence:

Listing 7.2: Ranking tests by failure history and co-change.

```

1 import pandas as pd
2
3 tests = pd.DataFrame({
4     "test": ["t_login", "t_export", "t_auth", "t_report", "t_api"],

```

```

5     "fail_rate": [0.40, 0.05, 0.30, 0.02, 0.10],
6     "cochange":  [0.8,  0.1,  0.6,  0.0,  0.4 ],
7     "distance":  [1,    4,    1,    5,    2   ], # graph hops to changed file
8 })
9 alpha, beta, gamma = 0.5, 0.3, 0.2
10 tests["score"] = alpha*tests.fail_rate + beta*tests.cochange - gamma*(tests.distance/5)
11 print(tests.sort_values("score", ascending=False)[["test", "score"]].to_string(index=False)
12 )
13 # Run top-2 first: t_login and t_auth catch most signal quickly.

```

7.6 Common Pitfalls

- **Leaking the future.** Training defect prediction on data that includes the bug-fix commit itself inflates AUC. Split strictly by time: train on sprints 1–8, validate on 9, test on 10.
- **Optimising the wrong metric.** AUC rewards ranking but ignores inspection cost. Always report effort-aware P_{opt} or Cost-Effectiveness alongside AUC.
- **Flaky tests as signal.** A test that flakes randomly will earn a high fail rate and dominate prioritisation. De-flake first or down-weight known flaky tests.
- **Estimation without intervals.** A point estimate of “5 days” that lands at 9 days breaks a sprint. Report P_{50} and P_{90} so planners can buffer realistically.

7.7 Closing the Loop: Measurement that Matters

Gates and predictions are only useful if they improve outcomes. The DORA framework (Forsgren et al.) gives four team-level yardsticks: **deployment frequency**, **lead time for changes**, **change failure rate**, and **mean time to recovery (MTTR)**. AI interventions should move at least one of these without harming the others — e.g., test prioritisation should raise deployment frequency (faster signal) without raising change failure rate (missed bugs). Always A/B the change: enable the ranker for half the PRs, compare escaped defects and review latency.

7.8 Chapter Summary

DevOps is a loop; AI shortens it. In CI/CD, ML reorders and prunes work so humans see what matters first. Defect prediction scores each change by $P(\text{bug} | x)$ using code, process, and semantic signals; effort estimation regresses from history to intervals that planning can actually use. Measure success with DORA metrics, not offline AUC alone.

7.9 Exercises

- E7.1 **Pipeline mapping.** Map each AI insertion from Sec. 7.2 to a DORA metric it should improve. For one, state a confound that could make the metric improve without real quality gain.
- E7.2 **Defect features.** You have only code metrics (M , LOC). A colleague proposes adding *number of authors* and *churn*. Explain why process features often beat code features, and design an experiment to test the claim on your repo.
- E7.3 **Threshold choice.** Your JIT model scores PRs. At $\tau = 0.5$, $P = 0.45$, $R = 0.70$; at $\tau = 0.7$, $P = 0.68$, $R = 0.40$. If reviewing a flagged PR costs 30 min and a missed bug costs 8 h, which τ minimises expected cost? Show work.
- E7.4 **Estimation evaluation.** You train an effort regressor with shuffled cross-validation and report MMRE = 0.18. Why is this optimistic? Redo the split temporally and explain which of MAE, MMRE, PRED(25) is most useful for sprint planning with buffers.
- E7.5 **Canary extension.** Extend Listing 7.1 to simulate canary analysis: generate two latency arrays (base-line vs. canary), run a Mann–Whitney U test, and auto-decide *rollback* vs. *promote*. Plot the latency distributions.

Chapter 8

The Future: Ethics, Collaboration, and Frontiers

Powerful tools demand wise hands. The future of AI in software engineering is not just what we can build, but what we should.

From zero: no ethics or IP background assumed. We build from everyday notions of fairness, ownership, and teamwork to the specific tensions AI brings to code, then map open research frontiers. Pictures anchor every idea.

Learning Outcomes. After this chapter you will be able to (1) name ethical risks of AI-generated code; (2) explain IP and licensing tensions; (3) contrast automation, augmentation, and collaboration modes; (4) sketch human–AI pair-programming patterns; (5) map current research frontiers and evaluate their maturity.

8.1 Intuition: Why “Just Ship It” Is Not Enough

An AI assistant can write a sorting routine in seconds. It can also write one that silently leaks user data, copies a GPL-licensed snippet without attribution, or reproduces a biased hiring filter from its training data. Speed without stewardship repeats the oldest failure in software: optimising for *can* while ignoring *should*.

This chapter treats that gap as engineering, not philosophy. For each concern we name the risk, show where it bites in the lifecycle, and give a concrete mitigation a team can adopt next sprint.

8.2 Ethics in AI-Generated Code

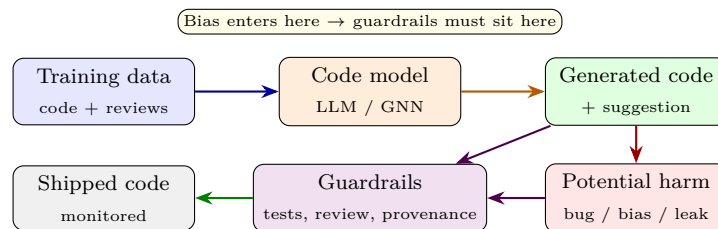
Four risks dominate practice:

- **Bias and fairness.** Training data reflect historical decisions — including biased ones. An AI code generator trained on Stack Overflow may suggest weaker input validation for certain locales, or a ranking model may

deprioritise bugs filed by non-native English speakers if their reports use different phrasing.

- **Safety and accountability.** Who is liable when auto-generated code causes an outage or a security breach? The developer who accepted the suggestion, the team that deployed it, or the model provider? Current practice places responsibility on the human committer, which means review obligations do not shrink just because a machine drafted the code.
- **Transparency.** Developers need to know *why* a suggestion was made and *what* sources influenced it. Black-box generation without provenance makes auditing impossible.
- **Environmental and labour costs.** Training large code models consumes significant energy; labelling data for review models relies on human annotators. Ethical adoption weighs these costs against gains.

Mitigations that work: diverse evaluation datasets that include under-represented bug types and locales, bias audits on flagged-vs.-accepted warnings across author demographics, mandatory human sign-off on generated security-critical code, and energy budgets in model selection (a smaller distilled model at 95% of the quality for 10% of the cost is often the responsible choice).



8.3 Intellectual Property and Licensing

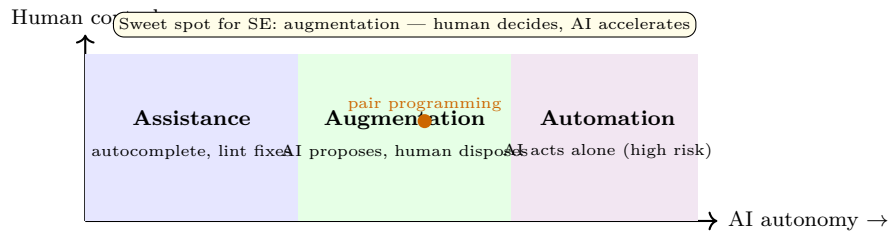
Code is intellectual property. When an LLM is trained on public repositories, it may memorise and regurgitate licensed code verbatim. Three tensions follow:

1. **Licence compliance.** GPL, AGPL, and other copyleft licences require derivative works to carry the same licence. Pasting an AI suggestion that reproduces GPL code into a proprietary codebase can create legal exposure.
2. **Attribution.** Many open-source licences (MIT, Apache-2.0) require attribution. If the model strips provenance, the user cannot comply.
3. **Ownership of generated code.** Is AI output owned by the user, the model provider, or no one? Jurisdictions differ, and terms of service vary — teams must read them rather than assume.

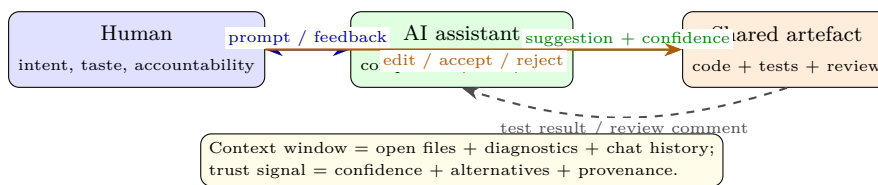
Practical hygiene: (i) enable provenance and licence filters (“block suggestions matching known GPL snippets above 80% similarity”); (ii) treat AI output like third-party code — scan it with a Software Composition Analysis (SCA) tool before merge; (iii) prefer models trained on permissively licensed data or with opt-out honoured; (iv) document AI assistance in commit messages for auditability. None of this replaces legal counsel, but it prevents the most common mistakes.

8.4 Human–AI Collaboration: Augmentation, Not Replacement

Three interaction modes describe how humans and AI share work:



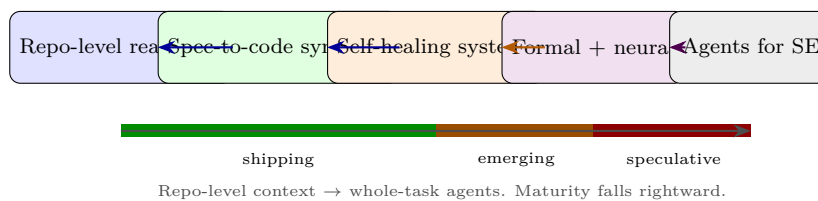
Pair-programming with AI is the dominant pattern today. The driver (human) states intent, the AI navigator suggests completions, tests, or refactors, and the human accepts, edits, or rejects. Good collaboration design follows three principles: **initiative balance** (AI suggests, human steers; either can ask “why?”), **common ground** (shared view of the codebase, open files, and recent errors), and **calibrated trust** (show confidence and alternatives, not just one answer).



Anti-patterns to avoid: *automation bias* (accepting suggestions without reading them), *deskilling* (novices who never learn to write a test because AI always does), and *accountability drift* (“the AI wrote it” as an excuse). Mitigate with mandatory reading time for AI diffs, learning-oriented prompts (“explain, don’t just fix”), and clear ownership rules.

8.5 Research Frontiers

Where is the field heading? Five frontiers are worth tracking, ordered from “shipping now” to “still speculative.”



1. **Repository-level reasoning.** From single-file completion to whole-repo understanding: cross-file references, build graphs, and issue history. Retrieval-augmented generation over the repo is the current workhorse.
2. **Specification-to-code synthesis.** Generate implementations from natural-language specs or formal contracts, with verification closing the loop (“generate and prove”).

3. **Self-healing systems.** Detect, diagnose, and patch failures autonomously in production, with humans approving patches that touch state or money.
4. **Neuro-symbolic SE.** Combine neural generation with symbolic guarantees: type checkers, model checkers, and program analysers as oracles that reject bad generations before a human sees them.
5. **Autonomous SE agents.** End-to-end agents that plan (decompose an issue into steps), act (edit, test, observe), and reflect (fix failures) over long horizons — evaluated on SWE-bench and similar benchmarks. Still brittle on large refactors.

Two evaluation shifts accompany these frontiers: from static benchmarks (HumanEval) to **interactive, repository-scale** tasks (SWE-bench, Defects4J with CI), and from accuracy alone to **cost, latency, and regret** (how much human time was actually saved, and at what risk?).

Example 8.1 Listing 8.1: Checking a generated snippet against a licence-blocked corpus.
(Provenance check for a generated snippet).

```

1 import difflib
2
3 BLOCKED_SNIPPETS = [
4     "def quicksort(arr):\n    if len(arr) <= 1:\n        return arr\n    pivot = arr[len(\n        arr)//2]", # toy GPL example
5 ]
6
7 def max_similarity(generated: str, corpus) -> float:
8     return max(difflib.SequenceMatcher(None, generated, s).ratio() for s in corpus)
9
10 generated = "def quicksort(arr):\n    if len(arr) <= 1:\n        return arr\n    pivot =\n        arr[len(arr)//2]\n    left = [x for x in arr if x < pivot]"
11 score = max_similarity(generated, BLOCKED_SNIPPETS)
12 print(f"max similarity to blocked corpus: {score:.2f}")
13 if score > 0.80:
14     print("BLOCK: too close to copyleft code -- rewrite or attribute")
15 else:
16     print("PASS: no close match")

```

8.6 Putting Ethics into Practice: A Checklist

Teams can operationalise this chapter with a short pre-merge checklist:

1. **Provenance check passed?** SCA scan for licence conflicts, similarity to blocked corpus below threshold.
2. **Bias audit clean?** Acceptance rate of AI suggestions does not differ by author locale or team at $p < 0.05$.
3. **Human sign-off present?** Every AI-generated change to auth, payments, or data handling has a human reviewer who can explain it.
4. **Energy budget respected?** Model choice justified against a smaller baseline; batch inference where possible.

A second listing extends provenance to embeddings for scale:

Listing 8.2: Scaled provenance with TF-IDF nearest neighbour.

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.metrics.pairwise import cosine_similarity
3
4 corpus = ["def quicksort(arr):\n    if len(arr)<=1: return arr", "def mergesort(a):\n
    ..."]
5 gen = "def quicksort(arr):\n    if len(arr)<=1: return arr\n    pivot=arr[0] "
6 vec = TfidfVectorizer().fit(corpus + [gen])
7 M = vec.transform(corpus + [gen])
8 sims = cosine_similarity(M[-1], M[:-1]).flatten()
9 print("max cosine:", sims.max(), "closest idx:", sims.argmax())
10 if sims.max() > 0.80:
11     print("flag for manual licence review")

```

8.7 Chapter Summary

Powerful code models amplify both capability and responsibility. Ethics demands guardrails for bias, safety, transparency, and cost; IP demands provenance and licence hygiene; collaboration demands augmentation designs where humans decide and AI accelerates. The frontier moves from file-level completion to repo-level agents with symbolic checks — and the measure of progress will be human time saved without regret.

8.8 Exercises

- E8.1 **Bias audit.** Your defect ranker flags bugs from one team $2\times$ more often than another at the same threshold. Propose a metric to detect this, a hypothesis for why it happens (training data? wording?), and a mitigation that preserves overall recall.
- E8.2 **Licence hygiene.** A generated 20-line function matches a GPL file at 85% similarity. List three lawful options for the team and one that is not. When would you choose rewriting over attribution?
- E8.3 **Collaboration design.** Sketch a pair-programming interaction where the AI suggests a fix with low confidence and two alternatives. What UI elements convey confidence and provenance without overwhelming the developer?
- E8.4 **Frontier maturity.** Place “self-healing patch for a null dereference in a Java microservice” on the maturity bar of Fig. 2. Justify with a concrete benchmark (Defects4J, SWE-bench) and state what would promote it from emerging to shipping.
- E8.5 **Provenance extension.** Extend Listing 8.1 to check against 100 snippets efficiently (hint: MinHash or embedding nearest-neighbour) and to report the closest match with its licence. Discuss false positives from boilerplate (e.g., getters).